Java ArrayList – Study Notes

What I learned from the JavaMastery course, plus the code I ran myself to check it.

1. What is an ArrayList?

- A class in `java.util` that implements the `List` interface.
- Stores elements in a **dynamic array** that grows automatically.
- Fast access by index.
- Slow insertion and deletion in the middle (elements must be shifted).
- **Not thread-safe.**

How it grows

A new ArrayList starts with a default capacity of `10`. When the internal array is full, Java creates a larger one (about 1.5× the old size in OpenJDK; the course says “1.5× or 2×”) and copies every existing element into it.

> That is why `add(element)` is O(1) on average: most adds are cheap, and only the occasional resize costs O(n).

2. Time complexity

Operation	Average	Worst	Why
`add(element)`	O(1)	O(n)	Fast unless a resize is needed
`add(index, element)`	O(n)	O(n)	Needs to shift elements
`get(index)`	O(1)	O(1)	Direct index access

Operation	Average	Worst	Why
`set(index, element)`	O(1)	O(1)	Direct overwrite
`remove(index)`	O(n)	O(n)	Shifts elements after deletion
`contains(element)`	O(n)	O(n)	Scans the list linearly
`size()`	O(1)	O(1)	Tracked internally

3. When not to use ArrayList

- You need frequent insertions or deletions in the middle or at the beginning.
- You work with very large data sets and want to avoid memory copies.

4. Basic operations I tested

```
// add
ArrayList<String> names = new ArrayList<>();
names.add("ahmad");
names.add("ali");
names.add("sara");
names.add(2, "omar");            // [ahmad, ali, omar, sara]

// get
names.get(2);                    // omar

// set (overwrites, does not shift)
names.set(1, "khaled");          // [ahmad, khaled, omar, sara]
```

5. The problem: removing inside a for loop

I expected this loop to empty the list. It doesn’t.

```
for (int i = 0; i < names.size(); i++) {
    names.remove(i);             // Index shifting problem
}
// size is 2, not 0
```

After `remove(i)`, every element to the right moves one position left. The next element takes the same index, but `i` moves on, so it is skipped. At the same time `size()` shrinks, so the loop ends early.

Try it: step through the loop



6. The solution: Iterator

```
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    it.next();
    it.remove();                 // safe removal
}
// size is 0
```

The iterator knows about the removal and keeps its position correct, so nothing is skipped.

> **Also remember:** calling `list.remove()` inside a for-each loop throws `ConcurrentModificationException`. Use `it.remove()` (or `removeIf` in Java 8+) instead.

7. Full runnable example

```
import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListDemo {
    public static void main(String[] args) {

        // add
        ArrayList<String> names = new ArrayList<>();
        names.add("ahmad");
        names.add("ali");
        names.add("sara");
        names.add(2, "omar");
        System.out.println("After add: " + names);

        // get
        System.out.println("get(2): " + names.get(2));

        // set
        names.set(1, "khaled");
        System.out.println("After set: " + names);

        // Problem: remove inside a regular for loop (Index Shifting)
        ArrayList<String> wrong = new ArrayList<>(names);
        System.out.println("size before: " + wrong.size());
        for (int i = 0; i < wrong.size(); i++) {
            wrong.remove(i);
        }
        System.out.println("size after (wrong): " + wrong.size());

        // Solution: Iterator
        ArrayList<String> correct = new ArrayList<>(names);
        Iterator<String> it = correct.iterator();
        while (it.hasNext()) {
            it.next();
            it.remove();
        }
        System.out.println("size after (Iterator): " + correct.size());
        System.out.println(correct.isEmpty() ? "Yes, it is empty" : "No,
is not");
    }
}
```

Output

```
After add: [ahmad, ali, omar, sara]
get(2): omar
After set: [ahmad, khaled, omar, sara]
size before: 4
size after (wrong): 2
size after (Iterator): 0
Yes, it is empty
```

Source: JavaMastery Academy course on Udemy. Code tested by me.